HTTP 安全標頭防護設定

一、簡介

HTTP 安全標頭(HTTP Security Headers)是網站伺服器於回應 HTTP 請求時所附加的控制項，可協助瀏覽器實施一系列安全政策，減少常見攻擊風險，如跨站指令碼(Cross-Site Scripting , XSS)、點擊劫持（Clickjacking)、網際網路媒體類型(Internet media type，亦稱 Multipurpose Internet Mail Extensions, MIME）混淆等，提升網站資安防護。

二、常見安全標頭設定

1. Content-Security-Policy(CSP)

限制瀏覽器可載入的資源來源，有效防止 XSS 攻擊。

功能簡介：

CSP 是一種網站安全機制，用來限制瀏覽器只允許從指定的可信來源載入或執行腳本或網頁資源（例如 JavaScript、圖片、樣式表等），藉此防止攻擊者在網頁中注入惡意腳本（跨站指令碼攻擊，XSS)。

實際防護效果：

- 若有攻擊者試圖將外部惡意 JavaScript 載入，瀏覽器會因 CSP 限制而阻止載入或執行，並在瀏覽器之開發者控制台顯示警告。
- 減少誤植或錯誤引用外部資源所造成的安全漏洞。

建議之設定值：

- 視個別系統站台而定，禁止非受信任之資源載入及腳本執行。
- 常見之設定指令 ：
    - default-src : 所有未特別指定的資源的預設來源
    - script-src : JavaScript 的來源（最重要）

- style-src : CSS、style 來源
- img-src : 圖片來源
- font-src : 字型來源
- connect-src : XHR, fetch API, WebSocket 等的連線來源
- media-src : video / audio 的來源

- 設定方式：
  - Content-Security-Policy default-src 'self' cdn.example.com;
  - 說明解釋：僅允許相同來源（same scheme, host and port）及 cdn.example.com 之資源
- 實務建議：

若 HTTP 回應標頭中的 Content-Security-Policy（CSP）設定不完整或僅部分實作，安全掃描工具在進行自動化分析時，可能會基於保守假設而判定潛在風險，進一步產生過度的風險評估結果。這類情況常見於僅設定部分資源限制（例如 default-src 未明確定義或缺少對特定來源的控管），導致工具誤以為應用程式允許來自任意來源的內容載入。

因此，CSP 應依實際應用架構與需求進行完整且明確的設定，以平衡安全防護需求與誤報風險。在實務上，若系統運作環境或資安稽核結果顯示 CSP 不完整僅屬設計考量而非漏洞，則可視具體情況調整或補充設定，以避免安全掃描工具產生不必要的高風險判定。

2. HTTP Strict-Transport-Security(HSTS)

強制瀏覽器透過 Https 連線，防止中間人攻擊。

功能簡介：

HTTP Strict-Transport-Security 為告知瀏覽器「**未來一段時間內，只允許使用 https 連線訪問網站**」的 HTTP 安全標頭。透過這個設定，能有效防止中間人攻擊（Man-In-The-Middle Attack, MITM）以及協議降級攻擊。

實際防護效果：

- 使用者瀏覽器收到此標頭，在設定的時間內，瀏覽器不再允許以 HTTP 連線到該網站。
- 即使使用者手動輸入 http://，瀏覽器也會自動改為 Https，防止流量被竊聽或篡改。

建議之設定值：

- Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
- max-age=31536000 為 1 年(最常用的安全建議值)，建議一開始不要鎖死太久，驗證穩定後再拉長，達到最終安全目標。
- includeSubDomains 表示 所有子網域也強制 HTTPS
- Preload 視情況設定，如增加 preload 屬性後，後續要提交到瀏覽器的 HSTS Preload 名單(hstspreload.org)

3. X-Content-Type-Options

防止瀏覽器猜測網際網路媒體類型(MIME)，減少攻擊面。

功能簡介：

X-Content-Type-Options 為 HTTP 回應標頭，告訴瀏覽器**不要嘗試去猜測(nosniff)檔案的 MIME 類型**，而是嚴格按照伺服器所指定的 Content-Type 來處理檔案。

實際防護效果：

- 有效避免瀏覽器將不安全的檔案誤判為可執行內容，降低 XSS 攻擊風險。
- 增強瀏覽器安全機制，確保內容以預期的方式被處理。

建議之設定值：

- X-Content-Type-Options: nosniff

4. X-Frame-Options

防止網站被嵌入於 iframe 中，降低點擊劫持攻擊(Clickjacking)風險。

功能簡介：

X-Frame-Options 為 HTTP 回應標頭，用來控制網站頁面是否允許被其他網站透過 <iframe> 或 <frame> 標籤嵌入。此標頭能有效防止「點擊劫持攻擊(Clickjacking)」。

實際防護效果：

- 有效避免惡意網站透過嵌入網站畫面，誘騙使用者誤點操作。
- 強化網站內容的顯示安全性，降低針對使用者的釣魚攻擊風險。

建議之設定值：

- X-Frame-Options: DENY
- 實際仍依網站系統需求而定如
    - X-Frame-Options: SAMEORIGIN 表示允許「相同網域」的頁面 iframe 嵌入
    - 如允許特定網域嵌入，建議改採 Content-Security-Policy(CSP)之設定指令：frame-ancestors

5. Referrer-Policy

控制瀏覽器在跳轉時傳送 referrer 的方式，保護用戶隱私。

功能簡介：

Referrer-Policy 是一個 HTTP 安全標頭，用來控制瀏覽器在用戶從一個網站跳轉到另一個網站時，如何傳送「來源網址」(referrer) 資訊，是否攜帶請求參數等資訊。

實際防護效果

- **減少敏感資訊外洩**
  防止 網址(URL)中包含的敏感資料（如身份驗證 Token、使用者 ID 或其他參數）被第三方網站獲取。

- **避免被惡意網站利用**
  避免攻擊者從 HTTP 頁面跳轉至不安全網站時，透過來源資訊蒐集用戶瀏覽行為。
- **提升用戶隱私保護**
  根據設定，限制來源資訊的詳細程度，讓用戶的瀏覽足跡不易被外洩或追蹤。

建議之設定值

- Referrer-Policy: strict-origin-when-cross-origin
- 說明解釋：同源傳完整的 URL，跨域僅傳網域（不送 path / query），降級連線（HTTPS → HTTP）不傳 Referer 標頭。

6. Permissions-Policy（前稱 Feature-Policy）

限制網站使用特定裝置功能，如麥克風、相機、定位等。

功能簡介

Permissions-Policy 用來限制或控制網站及其嵌入內容（如 iframe）對特定瀏覽器功能或裝置權限的使用。這些功能包括麥克風、相機、地理位置、全螢幕、感測器等。透過該標頭，網站可以指定哪些功能允許或禁止使用，提高安全性與隱私保護。

實際防護效果

- **防止未經授權的裝置存取**
  阻止惡意網站或第三方嵌入內容濫用裝置功能(如自動開啟相機、麥克風)，避免隱私洩露或監控風險。
- **減少攻擊面**
  限制功能使用，降低利用這些權限的攻擊途徑（如地理位置跟蹤、感測器攻擊)。
- **提升用戶信任與控制感**
  明確規範功能使用範圍，讓用戶在瀏覽時享有更好的隱私及安全保障。

建議之設定值

- Permissions-Policy: accelerometer=(), ambient-light-sensor=(), autoplay=(), battery=(), camera=(), clipboard-read=(), clipboard-write=(), display-capture=(), document-domain=(), encrypted-media=(), fullscreen=(), geolocation=(), gyroscope=(), magnetometer=(), microphone=(), midi=(), payment=(), picture-in-picture=(), publickey-credentials-get=(), sync-xhr=(), usb=(), vr=(), wake-lock=(), xr-spatial-tracking=() 為一條「預設全部關掉」指令。
- 常見支援之 Permissions Policy Features：
    - accelerometer　裝置加速度感測器
    - geolocation GPS 位置取得
    - camera　使用相機
    - microphone　使用麥克風
    - fullscreen　全螢幕模式
    - clipboard-read　讀取剪貼簿
    - clipboard-write 寫入剪貼簿
    - autoplay　自動播放多媒體
    - usb 存取 USB 裝置
    - payment Payment Request API
    - publickey-credentials-get　WebAuthn
    - vr / xr-spatial-tracking　WebXR API
    - display-capture 畫面錄製（如 getDisplayMedia）
- 設定方式：feature =（來源清單），一個允許只有同 origin 的 JS 可以使用 geolocation 之設如 Permissions-Policy: geolocation=('self')
- 實際視個別系統站台實際需求而設定，僅允許自身站台使用必要之瀏覽器 API，並確認關閉禁用不使用之相關瀏覽器 API
- 功能的實際預設狀態是由瀏覽器內建策略決定的，而非由 HTTP 標頭統一控制

三、檢測與輔助工具建議

利用檢測與輔助工具檢測網站是否正確設置安全標頭：

1、SecurityHeaders.com

- **功能**：快速掃描目標網站的 HTTP 回應標頭，檢查是否存在常見的安全標頭（如 CSP、HSTS、X-Frame-Options 等）。
- **特色**：操作簡單，只需輸入網址即可獲得標頭評分及詳細建議。
- **用途**：評估網站安全標頭的整體配置狀況，方便發現遺漏或配置錯誤。

2、Mozilla Observatory

- **功能**：由 Mozilla 提供的網站安全檢測工具，不僅檢查 HTTP 安全標頭，還涵蓋 SSL/TLS 配置及其他安全問題。
- **特色**：報告詳細，提供改善建議，並給予安全評分。
- **用途**：適合深入分析網站安全狀況，兼顧標頭及其他安全面向。

3、CSP Evaluator by Google

- **功能**：專門用來分析 Content-Security-Policy（CSP）設定的工具。
- **特色**：能評估 CSP 規則的有效性與安全性，檢測可能的漏洞或配置錯誤。
- **用途**：協助開發者優化 CSP 設定，增強防禦 XSS 攻擊的效果。

四、系統實作範例

在前述眾多 HTTP 安全標頭中，實務上開發人員應優先落實對資安風險影響最大的幾項設定。目前建議網站至少應該設定 6 個關鍵的 HTTP Security Headers，能大幅提高網站抵禦常見攻擊（如 XSS、Clickjacking、MIME 混淆、流量竊聽等）的能力，以下針對六個安全性優先級較高的 Response Header（Content-Security-Policy、HTTP Strict-Transport-Security、X-Content-Type-Options、X-Frame-Options、Referrer-Policy、Permissions-Policy），作為重點實作範例。本文將以 Java 與 .NET 兩種常見開發環境進行展示，示範如何在程式端設定這些標頭，並提供可直接套用的程式碼範例，協助開發人員快速

將 HTTP 安全標頭防護機制整合進既有系統。建議依實際系統情況調整各屬性與參數值，以強化安全性之防範。後續亦可類推至其他程式語言與框架，確保不同平台的網站應用均能達到一致的安全防護水準。

1. Java

在遵循 Java Servlet 規範 的應用伺服器（如 Tomcat、Jetty、WebSphere、WebLogic）上，最簡潔、可重用的做法是使用 Servlet Filter 並註冊啟用於服務器中(web.xml)，於回應階段統一加入安全標頭。

實作範例如下：

```
public class CommonHeader implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;

        res.setHeader("X-Frame-Options", "DENY");
        res.setHeader("Strict-Transport-Security", "max-age=31536000; includeSubDomains");
        res.setHeader("Content-Security-Policy", "script-src 'self' 'unsafe-inline' 'unsafe-eval'");
        res.setHeader("X-Content-Type-Options", "nosniff");
        res.setHeader("Referrer-Policy", "strict-origin-when-cross-origin");
        res.setHeader("Permissions-Policy", "camera=(), microphone=(), geolocation=(self), fullscreen=(self)");

        chain.doFilter(request, response);

    }

}
```

2. .NET

如果你的 .NET 專案是 ASP.NET（WebForms / MVC / .NET Core）跑在 IIS 上，可以直接在 web.config 加入 Response Headers。

實作範例如下：

```
<configuration>
        <system.webServer>
            <httpProtocol>
              <customHeaders>
                  <add name="X-Frame-Options" value="DENY" />
                  <add name="Strict-Transport-Security" value="max-age=31536000; includeSubDomains" />
                  <add name="Content-Security-Policy" value="script-src 'self' 'unsafe-inline' 'unsafe-eval'" />
                  <add name="X-Content-Type-Options" value="nosniff" />
                  <add name="Referrer-Policy" value="strict-origin-when-cross-origin" />
                  <add name="Permissions-Policy" value="camera=(), microphone=(), geolocation=(self), fullscreen=(self)"
/>
              </customHeaders>
            </httpProtocol>
        </system.webServer>
    </configuration>
```

五、注意事項：

透過線上工具 SecurityHeaders.com 檢測設有 6 個關鍵的 HTTP Security Headers 之網站伺服器，將獲得一 A 級以上之安全網站檢測評分(大部分核心標頭已設定，僅少部分設定值未涵蓋，或可再強化)。

HTTP Security Headers 之屬性與值，建議依專案實際情況增修或強化，實際之使用將依專案設計架構考量而強化，否則可能會導致系統無法正常運行之情況，如 Content-Security-Policy 表頭建議最高安全設定回復值為「Content-Security-Policy: default-src 'self'」僅允許瀏覽器載入或執行來自同站台之腳本與資源，再根據各資源需求放寬設定(script-src 、img-src 、……)，但考量現代系統使用前後端分離之架構與框架之深度差異，常將前端可執行之腳本撰寫內嵌在網頁內部，並可能使用非預先編譯之瀏覽器端模板編譯語法(未使用前端建構工具方式開發)；在檔案存儲方面，將檔案、影像存放在雲端空間或透過 CDN 服務存取等，因此考量專早期案架構之設計，仍必須開放外部檔案伺服器及瀏覽器執行內嵌腳本等，開放設定如「Content-Security-Policy: script-src 'self' 'unsafe-inline'; img-src 'self' https://*.cdn.net https://my-bucket.s3.amazonaws.com……」將造成弱點掃瞄器判斷出現其他中高風險弱點之可能，各系統請依自身實際情況，自行設定並評估取捨。

Security Report Summary

A

Site:	https://nrcsa[redacted]
IP Address:	[redacted]
Report Time:	27 Aug 2025 08:37:58 UTC
Headers:	✔ X-Frame-Options ✔ Strict-Transport-Security ✔ Content-Security-Policy ✔ X-Content-Type-Options ✔ Referrer-Policy ✔ Permissions-Policy
Warning:	Grade capped at A, please see warnings below.
Advanced:	Great grade! Perform a deeper security analysis of your website and APIs: Try Now

Warnings

Content-Security-Policy	This policy contains 'unsafe-inline' which is dangerous in the script-src directive. This policy contains 'unsafe-eval' which is dangerous in the script-src directive.

Raw Headers

HTTP/1.1	200
Access-Control-Allow-Origin	*
Access-Control-Allow-Methods	GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers	Authorization, Content-Type, X-Requested-With
Access-Control-Allow-Credentials	true
Access-Control-Max-Age	3600
X-Frame-Options	**DENY**
Strict-Transport-Security	**max-age**=31536000; **includeSubDomains**
Content-Security-Policy	**script-src** 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options	**nosniff**
Referrer-Policy	**strict-origin-when-cross-origin**
Permissions-Policy	**camera**=(), **microphone**=(), **geolocation**=(self), **fullscreen**=(self)
Content-Type	text/html;charset=utf-8
Content-Language	en-GB